

GITHUB LINK:

<https://github.com/DHAANIBAHL/Gaze-based-interface-for-physically-disabled-users>

Code snippets:

1. Understanding the data

```
clc;
clear;
sampleFile = 'C:\Users\namra\MPIIGaze\Data\Normalized\p00\Day01.mat';
data = load(sampleFile);
whos('-file', sampleFile)
S = load('C:\Users\namra\MPIIGaze\Data\Normalized\p00\Day01.mat');
fieldnames(S.data)
size(S.data.right)
size(S.data.left)
class(S.data.right)
class(S.data.left)
```

2. Preprocessing

```
clc;
clear;
close all;
root = 'C:\Users\namra\MPIIGaze\Data\Normalized';
imgSize = [64 64];
X = zeros(64,64,0,'double');
Y = zeros(0,3);
subs = dir(fullfile(root,'p*'));
subs = subs([subs.isdir]);
for s = 1:length(subs)
    subjPath = fullfile(root, subs(s).name);
    dayFiles = dir(fullfile(subjPath,'Day*.mat'));
    for d = 1:length(dayFiles)
        S = load(fullfile(subjPath, dayFiles(d).name));
        eyes = {'right','left'};
        for e = 1:2
            E = S.data.(eyes{e});
            imgs = E.image; % N x 36 x 60
            gaze = E.gaze; % N x 3
            N = size(imgs,1);
            for i = 1:N
                img = squeeze(imgs(i,:,:)); % 36x60
                img = imresize(img, imgSize);
                img = im2double(img);
            end
        end
    end
end
```

```

        X(:, :, end+1) = img;    %  NOW WORKS
        Y(end+1, :) = gaze(i, :);
    end
end
end
end
disp('Final dataset size:')
disp(size(X))
disp(size(Y))
save('X_mpiigaze_normalized.mat', 'X', '-v7.3')
save('Y_mpiigaze_gaze.mat', 'Y')

```

3. ML training code

```
from google.colab import drive
```

1. Mount the drive at the specified path

```
drive.mount('/content/drive')
```

```
print("\nGoogle Drive mounted successfully. Files can now be accessed.")
```

2. Verify Dataset Directory

```
import os
```

```
DATASET_PATH = "/content/drive/MyDrive/Gaze detection/"
```

Check if path exists

```
if os.path.exists(DATASET_PATH):
    print("Dataset directory found.\n")
    print("Contents of the dataset directory:")
    for item in os.listdir(DATASET_PATH):
        print(f" - {item}")
else:
```

```
    print("Dataset directory not found. Please check the path.")
```

3. Check Individual Dataset Files

```
import os
```

```
X_FILE_PATH = "/content/drive/MyDrive/Gaze detection/X_mpiigaze_normalized.mat"
```

```
Y_FILE_PATH = "/content/drive/MyDrive/Gaze detection/Y_mpiigaze_gaze.mat"
```

Check if each file exists and print a clear message

```
if os.path.exists(X_FILE_PATH):
    print(f"Found X dataset file: {X_FILE_PATH}")
else:
```

```

    print(f"X dataset file not found at: {X_FILE_PATH}")

if os.path.exists(Y_FILE_PATH):
    print(f"Found Y dataset file: {Y_FILE_PATH}")
else:
    print(f"Y dataset file not found at: {Y_FILE_PATH}")

# 4. Inspect Dataset Files

import os
import h5py
import numpy as np

# Paths to the dataset files
X_PATH = "/content/drive/MyDrive/Gaze detection/X_mpiigaze_normalized.mat"
Y_PATH = "/content/drive/MyDrive/Gaze detection/Y_mpiigaze_gaze.mat"

# 4a. Check size of Y dataset file
y_size_bytes = os.path.getsize(Y_PATH)
y_size_mb = y_size_bytes / (1024 * 1024)
print(f"Size of Y dataset file: {y_size_mb:.2f} MB")

# 4b. Open the X dataset file using h5py
X_file = h5py.File(X_PATH, 'r')

# List all datasets/keys inside the X file
x_keys = list(X_file.keys())
print("\nDatasets available inside X file:")
for key in x_keys:
    print(f" - {key}")

# 5. Load and Inspect Dataset Shapes

import scipy.io

# ---- Load X dataset ----
X = X_file['X'] # Access the dataset inside the .mat file
print(f"Shape of X dataset: {X.shape}")

# ---- Load Y dataset ----
Y_path = "/content/drive/MyDrive/Gaze detection/Y_mpiigaze_gaze.mat"
Y_data = scipy.io.loadmat(Y_path)

print("\nKeys inside Y dataset file:")

```

```

for key in Y_data.keys():
    print(f" - {key}") # just to confirm keys

# Extract the actual Y data
Y = Y_data['Y'] # Ensure this key matches
print(f"\nShape of Y dataset: {Y.shape}")

with h5py.File(X_PATH, 'r') as X_file:
    print("Available datasets:", list(X_file.keys()))
    print("Shape of X:", X_file['X'].shape)

# 6. Visualize Sample Gaze Data
import matplotlib.pyplot as plt
import random

# ---- Visualize a random selection of 25 gaze images ----
num_samples_to_show = 25
indices = random.sample(range(X.shape[0]), num_samples_to_show)

plt.figure(figsize=(10,10))
for i, idx in enumerate(indices):
    plt.subplot(5,5,i+1)
    plt.imshow(X[idx], cmap='gray') # assuming grayscale images
    plt.axis('off')
plt.suptitle("Sample Gaze Images from X Dataset", fontsize=16)
plt.show()

# ---- Visualize corresponding gaze points from Y ----
# Assuming Y has shape (num_samples, 2) with (x, y) coordinates
sample_Y = Y[indices]

plt.figure(figsize=(6,6))
plt.scatter(sample_Y[:,0], sample_Y[:,1], c='red', alpha=0.7)
plt.title("Gaze Coordinates from Y Dataset", fontsize=14)
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.grid(True)
plt.show()

# 7. Visualize Gaze Coordinates Distribution

```

```

import numpy as np

num_samples = 20000
indices = np.random.choice(X.shape[0], num_samples, replace=False)
indices = np.sort(indices)

X_sample = X[indices].astype("float32")
Y_sample = Y[indices]

print(f'Sampled X shape: {X_sample.shape}')
print(f'Sampled Y shape: {Y_sample.shape}')
import matplotlib.pyplot as plt
import seaborn as sns

# Scatter plot of all sampled gaze points
plt.figure(figsize=(6,6))
plt.scatter(Y_sample[:,0], Y_sample[:,1], c='blue', alpha=0.3, s=10)
plt.title("Scatter of Gaze Coordinates (Y_sample)", fontsize=14)
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.grid(True)
plt.show()

# Heatmap / density of gaze points
plt.figure(figsize=(6,6))
sns.kdeplot(
    x=Y_sample[:,0], y=Y_sample[:,1],
    fill=True, cmap='Reds', bw_adjust=0.5, thresh=0.05
)
plt.title("Density Heatmap of Gaze Coordinates", fontsize=14)
plt.xlabel("X Coordinate")
plt.ylabel("Y Coordinate")
plt.show()

# Optional: histogram for X and Y coordinates separately
plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.hist(Y_sample[:,0], bins=50, color='skyblue', edgecolor='black')
plt.title("Histogram of X Coordinates")
plt.subplot(1,2,2)
plt.hist(Y_sample[:,1], bins=50, color='salmon', edgecolor='black')
plt.title("Histogram of Y Coordinates")
plt.show()

```

8. Explore and Split Sampled Dataset

```
import numpy as np
```

```
from sklearn.model_selection import train_test_split
```

Check the range of values in X_sample

```
print(f" ♦ X_sample range: min = {X_sample.min():.4f}, max = {X_sample.max():.4f}")
```

Split dataset into training and testing sets

```
X_train, X_test, Y_train, Y_test = train_test_split(
    X_sample, Y_sample,
    test_size=0.2, # 20% for testing
    random_state=42 # ensures reproducibility
)
```

---- Print shapes of all splits ----

```
print(f"X_train shape: {X_train.shape}")
```

```
print(f"Y_train shape: {Y_train.shape}")
```

```
print(f"X_test shape: {X_test.shape}")
```

```
print(f"Y_test shape: {Y_test.shape}")
```

9. Inspect Y_sample

```
import numpy as np
```

```
import sys
```

```
import tensorflow as tf
```

Check the average magnitude of gaze vectors

```
mean_norm = np.mean(np.linalg.norm(Y_sample, axis=1))
```

```
print(f" ♦ Average magnitude of gaze vectors in Y_sample: {mean_norm:.4f}")
```

Upgrade pip

```
!{sys.executable} -m pip install --upgrade pip
```

Install TensorFlow (specific version)

```
!{sys.executable} -m pip install tensorflow
```

Uninstall any conflicting versions

```
!pip uninstall -y tensorflow tensorflow-cpu tensorboard
```

```
!pip uninstall -y tensorflow tensorflow-cpu tensorboard
```

Install desired version

```
!pip install tensorflow==2.20.0
```

Import TensorFlow and check version

```
import tensorflow as tf
print(f"TensorFlow version: {tf.__version__}")
```

10. Normalize Gaze Vectors for Training

```
import numpy as np
import tensorflow as tf
```

```
# ---- Normalize Y_train and Y_test ----
```

```
# Each gaze vector is scaled to unit length
```

```
Y_train_norm = Y_train / np.linalg.norm(Y_train, axis=1, keepdims=True)
```

```
Y_test_norm = Y_test / np.linalg.norm(Y_test, axis=1, keepdims=True)
```

```
# ---- Verify normalization ----
```

```
print(f" ♦ Example normalized gaze vector (first sample): {Y_train_norm[0]}")
```

```
print(f" ♦ Norm of first sample vector: {np.linalg.norm(Y_train_norm[0]):.4f}")
```

11. Build CNN Model for Gaze Prediction

```
from tensorflow.keras import layers, models
import tensorflow as tf
```

```
# Sequential CNN architecture
```

```
# Input: 64x64 grayscale eye image
```

```
# Output: 3D normalized gaze vector
```

```
model = models.Sequential([
```

```
    # First convolution block
```

```
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(64,64,1)),
```

```
    layers.MaxPooling2D(2,2),
```

```
    # Second convolution block
```

```
    layers.Conv2D(64, (3,3), activation='relu'),
```

```
    layers.MaxPooling2D(2,2),
```

```
    # Third convolution block
```

```
    layers.Conv2D(128, (3,3), activation='relu'),
```

```
    layers.MaxPooling2D(2,2),
```

```
    # Flatten feature maps
```

```
    layers.Flatten(),
```

```

# Fully connected layer
layers.Dense(128, activation='relu'),

# Output 3D gaze vector
layers.Dense(3),

# Normalize output to unit vector
layers.Lambda(lambda x: tf.math.l2_normalize(x, axis=1))
])

# 12. Define Cosine Similarity Loss

# Loss = 1 - cosine similarity
# Since outputs are normalized, this directly penalizes angular difference
def cosine_loss(y_true, y_pred):
    y_true = tf.math.l2_normalize(y_true, axis=1)
    y_pred = tf.math.l2_normalize(y_pred, axis=1)
    return 1 - tf.reduce_sum(y_true * y_pred, axis=1)

# 13. Compile the Model

from tensorflow.keras import optimizers

model.compile(
    optimizer=optimizers.Adam(learning_rate=1e-4),
    loss=cosine_loss
)

# 14. Train the Model

history = model.fit(
    X_train,
    Y_train_norm,
    validation_data=(X_test, Y_test_norm),
    epochs=25,
    batch_size=64,
    verbose=1
)

# 15. Visualize Training vs Validation Loss

```



```

import matplotlib.pyplot as plt

plt.figure()
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.xlabel("Epoch")
plt.ylabel("Cosine Loss")
plt.title("Training vs Validation Cosine Loss")
plt.legend(["Train Loss", "Validation Loss"])
plt.show()

```

16. Evaluate Model using Angular Error

```

import numpy as np
import matplotlib.pyplot as plt

# ---- Generate Predictions ----
Y_pred = model.predict(X_test)

# ---- Compute Cosine Similarity ----
# Dot product between predicted and true normalized vectors
dot = np.sum(Y_pred * Y_test_norm, axis=1)

# Numerical stability (avoid arccos domain error)
dot = np.clip(dot, -1.0, 1.0)

# ---- Convert to Angular Error in Degrees ----
angles = np.degrees(np.arccos(dot))

# ---- Print Evaluation Metrics ----
print("Mean Angular Error (degrees):", np.mean(angles))
print("Median Angular Error (degrees):", np.median(angles))

```

17. Visualize Angular Error Distribution

```

plt.figure()
plt.hist(angles, bins=30)
plt.xlabel("Angular Error (Degrees)")
plt.ylabel("Frequency")
plt.title("Distribution of Angular Error on Test Set")

```

```
plt.show()
```

18. Inspect Sample Predictions vs Ground Truth

```
# ---- Predict first 5 test samples ----
```

```
pred = model.predict(X_test[:5])
```

```
print("Predictions:")
```

```
print(pred)
```

```
print("\nGround Truth:")
```

```
print(Y_test[:5])
```

```
# ---- Compare Using Cosine Similarity ----
```

```
# Normalize ground truth to match model output format
```

```
true_norm = Y_test[:5] / np.linalg.norm(Y_test[:5], axis=1, keepdims=True)
```

```
dot = np.sum(pred * true_norm, axis=1)
```

```
dot = np.clip(dot, -1.0, 1.0)
```

```
angles = np.degrees(np.arccos(dot))
```

```
print("\nAngular Errors (degrees) for these 5 samples:")
```

```
print(angles)
```

19. Plot Training and Validation Loss

```
import matplotlib.pyplot as plt
```

```
plt.figure()
```

```
# Plot training loss
```

```
plt.plot(history.history['loss'], label='Train Loss')
```

```
# Plot validation loss
```

```
plt.plot(history.history['val_loss'], label='Validation Loss')
```

```
plt.xlabel("Epoch")
```

```
plt.ylabel("Cosine Loss")
```

```
plt.title("Training vs Validation Loss Curve")
```

```
plt.legend()
```

```
plt.grid(True)
```

```
plt.show()
```

```
# 20. Inspect Raw Loss Values
```

```
print("Training Loss per Epoch:")  
print(history.history['loss'])
```

```
print("\nValidation Loss per Epoch:")  
print(history.history['val_loss'])
```

```
# 21. Additional Performance Metrics
```

```
import numpy as np
```

```
# Ensure predictions are normalized  
Y_pred = model.predict(X_test)  
Y_pred = Y_pred / np.linalg.norm(Y_pred, axis=1, keepdims=True)
```

```
# Normalize ground truth  
Y_true = Y_test / np.linalg.norm(Y_test, axis=1, keepdims=True)
```

```
# ---- Cosine Similarity ----  
cos_sim = np.sum(Y_pred * Y_true, axis=1)
```

```
mean_cos_sim = np.mean(cos_sim)
```

```
# ---- Angular Error ----  
cos_sim = np.clip(cos_sim, -1.0, 1.0)  
angles = np.degrees(np.arccos(cos_sim))
```

```
mean_angle = np.mean(angles)  
median_angle = np.median(angles)
```

```
# ---- RMSE (vector space) ----  
rmse = np.sqrt(np.mean((Y_pred - Y_true) ** 2))
```

```
# ---- Percentage within threshold ----  
within_10 = np.mean(angles < 10) * 100  
within_20 = np.mean(angles < 20) * 100
```

```

within_30 = np.mean(angles < 30) * 100

print("Mean Cosine Similarity:", mean_cos_sim)
print("Mean Angular Error (deg):", mean_angle)
print("Median Angular Error (deg):", median_angle)
print("RMSE:", rmse)
print("% within 10°:", within_10)
print("% within 20°:", within_20)
print("% within 30°:", within_30)

print(history.history['loss'])
print(history.history['val_loss'])

```

4. Frontend

```

Index.html
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Eye Tracking Mouse Dashboard</title>
  </head>
  <body>
    <div id="app"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>

```

```

APP.JSX
import React from "react";
import ReactDOM from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import App from "./App";
import "./styles/globals.css";

ReactDOM.createRoot(document.getElementById("app")).render(
  <React.StrictMode>
    <BrowserRouter>
      <App />
    </BrowserRouter>
  </React.StrictMode>
);

```

MAIN.JSX

```
import { AnimatePresence } from "framer-motion";
import { Navigate, Route, Routes, useLocation } from "react-router-dom";
import MainLayout from "../layout/MainLayout";
import AccuracyTest from "../pages/AccuracyTest";
import Calibration from "../pages/Calibration";
import EyeControl from "../pages/EyeControl";
import Home from "../pages/Home";

function App() {
  const location = useLocation();

  return (
    <AnimatePresence mode="wait">
      <Routes location={location} key={location.pathname}>
        <Route path="/" element={<MainLayout />} />
        <Route index element={<Home />} />
        <Route path="calibration" element={<Calibration />} />
        <Route path="control" element={<EyeControl />} />
        <Route path="accuracy" element={<AccuracyTest />} />
        <Route path="*" element={<Navigate to="/" replace />} />
      </Route>
    </Routes>
  </AnimatePresence>
  );
}

export default App;
```

[API.JS](#)

```
const API_BASE_URL = import.meta.env.VITE_API_BASE_URL || "http://X.X.X.X";

async function request(path, options = {}) {
  const response = await fetch(`${API_BASE_URL}${path}`, {
    headers: {
      "Content-Type": "application/json",
      ...(options.headers || {}),
    },
    ...options,
  });
}
```

```

let data = null;
try {
  data = await response.json();
} catch (_error) {
  data = null;
}

if (!response.ok) {
  const message = data?.error || `Request failed (${response.status})`;
  throw new Error(message);
}

return data;
}

export function getHealth() {
  return request("/health");
}

export function getStatus() {
  return request("/api/status");
}

export function getAvailableCameras() {
  return request("/api/cameras");
}

export function startTracking(payload) {
  return request("/api/tracking/start", {
    method: "POST",
    body: JSON.stringify(payload || {}),
  });
}

export function sendTrackingFrame(imageDataUrl) {
  return request("/api/tracking/frame", {
    method: "POST",
    body: JSON.stringify({ image: imageDataUrl }),
  });
}

export function stopTracking() {
  return request("/api/tracking/stop", { method: "POST" });
}

```

```
}
```

```
export function saveSettings(settings) {  
  return request("/api/settings", {  
    method: "POST",  
    body: JSON.stringify(settings || {}),  
  });  
}
```

```
export function startCalibrationSession() {  
  return request("/api/calibration/start", { method: "POST" });  
}
```

```
export function sendCalibrationFrame(imageDataUrl, targetIndex) {  
  return request("/api/calibration/frame", {  
    method: "POST",  
    body: JSON.stringify({ image: imageDataUrl, target_index: targetIndex }),  
  });  
}
```

```
export function completeCalibrationSession() {  
  return request("/api/calibration/complete", { method: "POST" });  
}
```

```
export function submitAccuracyReport(report) {  
  return request("/api/accuracy/report", {  
    method: "POST",  
    body: JSON.stringify(report || {}),  
  });  
}
```

```
export function getTrackingLog() {  
  return request("/api/tracking/log");  
}
```

HOME.JSX

```
import { motion } from "framer-motion";  
import { useNavigate } from "react-router-dom";
```

```
function Home() {  
  const navigate = useNavigate();  
  
  return (  
    <motion.div
```

```

initial={{ opacity: 0, y: 12 }}
animate={{ opacity: 1, y: 0 }}
exit={{ opacity: 0, y: -8 }}
transition={{ duration: 0.35, ease: "easeOut" }}
className="hero-ui-shell"
>
<section className="hero-ui-page">
  <div className="hero-ui-content">
    <div className="hero-ui-left">
      <p className="hero-ui-tagline">Unlock the Power of Your Gaze</p>
      <h1 className="hero-ui-title">
        Hands-free <span>Control</span>
        <br />
        with <span>smart eye</span>-tracking.
      </h1>
      <p className="hero-ui-description">
        Experience hands-free digital control with advanced AI-driven gaze
        tracking. Calibrate in seconds and navigate apps, communication boards
        and the web using only your eyes.
      </p>
      <button
        type="button"
        className="hero-ui-cta"
        onClick={() => navigate("/calibration")}
      >
        Start Calibration
      </button>
      <p className="hero-ui-scroll-hint">Scroll down to continue</p>
    </div>

    <div className="hero-ui-right">
      <div className="hero-ui-orbit" />
      
      <div className="hero-ui-mini-panel">
        <div className="hero-ui-mini-line" />
        <div className="hero-ui-mini-line" />
        <div className="hero-ui-mini-line" />
        <div className="hero-ui-mini-line" />
      </div>
      <article className="hero-ui-quick-card">

```



```

        <h2>Quick calibration</h2>
        <p>
            Start a 10-second calibration to personalize gaze mapping for
            accurate control across lighting conditions.
        </p>
    </article>
</div>
</div>
</section>
</motion.div>
);
}

```

```
export default Home;
```

```
EYECONTROL.JSX
```

```

import { motion } from "framer-motion";
import { useNavigate } from "react-router-dom";
import GlassCard from "../components/GlassCard";
import GradientButton from "../components/GradientButton";
import SliderControl from "../components/SliderControl";
import StatusIndicator from "../components/StatusIndicator";
import { useEffect, useRef, useState } from "react";
import {
    getAvailableCameras,
    getTrackingLog,
    getStatus,
    saveSettings,
    sendTrackingFrame,
    startTracking,
    stopTracking,
} from "../services/api";

```

```

function EyeControl() {
    const navigate = useNavigate();
    const videoRef = useRef(null);
    const canvasRef = useRef(null);
    const mediaStreamRef = useRef(null);
    const autoSaveTimeoutRef = useRef(null);
    const frameIntervalRef = useRef(null);
    const frameInflightRef = useRef(false);
    const [paused, setPaused] = useState(true);
    const [sensitivity, setSensitivity] = useState(68);
    const [speed, setSpeed] = useState(72);

```

```

const [blinkThreshold, setBlinkThreshold] = useState(38);
const [smoothing, setSmoothing] = useState(56);
const [edgeBoost, setEdgeBoost] = useState(35);
const [source, setSource] = useState("webcam");
const [cameraIndex, setCameraIndex] = useState(0);
const [ipUrl, setIpUrl] = useState("http://127.0.0.1:8080/shot.jpg");
const [loading, setLoading] = useState(false);
const [apiStatus, setApiStatus] = useState("Checking backend...");
const [backendOnline, setBackendOnline] = useState(false);
const [availableCameras, setAvailableCameras] = useState([]);
const [recentLog, setRecentLog] = useState("");
const [previewReady, setPreviewReady] = useState(false);
const [previewError, setPreviewError] = useState("");

const confidence = paused ? "0%" : `${Math.round((sensitivity + speed) / 2)}%`;

const stopPreview = () => {
  if (mediaStreamRef.current) {
    mediaStreamRef.current.getTracks().forEach((track) => track.stop());
    mediaStreamRef.current = null;
  }
  if (videoRef.current) {
    videoRef.current.srcObject = null;
  }
  setPreviewReady(false);
};

const stopFrameStreaming = () => {
  if (frameIntervalRef.current) {
    window.clearInterval(frameIntervalRef.current);
    frameIntervalRef.current = null;
  }
  frameInflightRef.current = false;
};

const startFrameStreaming = () => {
  if (frameIntervalRef.current || source !== "webcam") {
    return;
  }
  frameIntervalRef.current = window.setInterval(async () => {
    if (frameInflightRef.current || paused) {
      return;
    }
  }, 1000);
  const video = videoRef.current;

```

```

const canvas = canvasRef.current;
if (!video || !canvas || video.readyState < 2) {
  return;
}
const width = video.videoWidth || 640;
const height = video.videoHeight || 480;
canvas.width = width;
canvas.height = height;
const context = canvas.getContext("2d");
if (!context) {
  return;
}
context.drawImage(video, 0, 0, width, height);
const dataUrl = canvas.toDataURL("image/jpeg", 0.62);

try {
  frameInflightRef.current = true;
  const response = await sendTrackingFrame(dataUrl);
  if (!response.detected) {
    setApiStatus("Tracking running, but no face/eyes detected.");
  }
} catch (error) {
  setApiStatus(error.message || "Failed to stream webcam frame.");
} finally {
  frameInflightRef.current = false;
}
}, 70);
};

const startPreview = async () => {
  if (source !== "webcam" || !navigator.mediaDevices?.getUserMedia) {
    stopPreview();
    return;
  }
  try {
    stopPreview();
    setPreviewError("");
    const stream = await navigator.mediaDevices.getUserMedia({
      video: { width: { ideal: 1280 }, height: { ideal: 720 } },
      audio: false,
    });
    mediaStreamRef.current = stream;
    if (videoRef.current) {
      videoRef.current.srcObject = stream;
    }
  }

```

```

        await videoRef.current.play();
    }
    setPreviewReady(true);
} catch (_error) {
    setPreviewReady(false);
    setPreviewError(
        "Browser camera preview unavailable. Allow camera permission in browser."
    );
}
};

```

```

useEffect(() => {
    let mounted = true;
    const loadStatus = async () => {
        try {
            const status = await getStatus();
            if (!mounted) {
                return;
            }
            setBackendOnline(true);
            setPaused(!status.running);
            setSource(status.source || "webcam");
            setIpUrl(status.url || "http://127.0.0.1:8080/shot.jpg");
            setCameraIndex(status.camera_index ?? 0);
            setApiStatus(
                status.last_error ||
                (status.running ? "Tracking process is running." : "Tracker is idle.")
            );
            if (status.settings) {
                setSensitivity(status.settings.sensitivity ?? 68);
                setSpeed(status.settings.speed ?? 72);
                setBlinkThreshold(status.settings.blinkThreshold ?? 38);
                setSmoothing(status.settings.smoothing ?? 56);
                setEdgeBoost(status.settings.edgeBoost ?? 35);
            }
        } catch (_error) {
            if (!mounted) {
                return;
            }
            setBackendOnline(false);
            setPaused(true);
            setApiStatus("Backend unavailable. Start backend/server.py first.");
        }
    };
};

```

```

loadStatus();
const intervalId = window.setInterval(loadStatus, 2500);
return () => {
  window.clearInterval(intervalId);
  mounted = false;
};
}, []);

```

```

useEffect(() => {
  let active = true;
  const loadCameras = async () => {
    try {
      const response = await getAvailableCameras();
      if (!active) {
        return;
      }
      const cameras = response?.cameras || [];
      setAvailableCameras(cameras);
      if (cameras.length > 0 && !cameras.includes(cameraIndex)) {
        setCameraIndex(cameras[0]);
      }
    } catch (_error) {
      if (active) {
        setAvailableCameras([]);
      }
    }
  };
  loadCameras();
  return () => {
    active = false;
  };
}, []);

```

```

useEffect(() => {
  let mounted = true;
  const loadLog = async () => {
    try {
      const response = await getTrackingLog();
      if (!mounted) {
        return;
      }
      const lines = response?.lines || [];
      setRecentLog(lines.slice(-1)[0] || "");
    } catch (_error) {

```

```

    if (mounted) {
      setRecentLog("");
    }
  }
};
loadLog();
const intervalId = window.setInterval(loadLog, 3000);
return () => {
  mounted = false;
  window.clearInterval(intervalId);
};
}, []);

```

```

useEffect(() => {
  if (source === "webcam") {
    startPreview();
  } else {
    stopPreview();
    stopFrameStreaming();
  }
}, [source]);

```

```

useEffect(() => {
  return () => {
    stopPreview();
    stopFrameStreaming();
  };
}, []);

```

```

useEffect(() => {
  if (!paused && source === "webcam" && previewReady) {
    startFrameStreaming();
  } else {
    stopFrameStreaming();
  }
  return () => {
    stopFrameStreaming();
  };
}, [paused, source, previewReady]);

```

```

useEffect(() => {
  if (!backendOnline) {
    return;
  }
}

```

```

    if (autoSaveTimeoutRef.current) {
      window.clearTimeout(autoSaveTimeoutRef.current);
    }
    autoSaveTimeoutRef.current = window.setTimeout(() => {
      saveSettings({
        sensitivity,
        speed,
        blinkThreshold,
        smoothing,
        edgeBoost,
      }).catch(() => {});
    }, 500);
    return () => {
      if (autoSaveTimeoutRef.current) {
        window.clearTimeout(autoSaveTimeoutRef.current);
      }
    };
  }, [backendOnline, sensitivity, speed, blinkThreshold, smoothing, edgeBoost]);

```

```

const handleSaveSettings = async () => {
  try {
    setLoading(true);
    await saveSettings({
      sensitivity,
      speed,
      blinkThreshold,
      smoothing,
      edgeBoost,
    });
    setApiStatus("Settings saved to backend.");
  } catch (error) {
    setApiStatus(error.message);
  } finally {
    setLoading(false);
  }
};

```

```

const handleToggleTracking = async () => {
  try {
    setLoading(true);
    if (paused) {
      if (source === "webcam" && !mediaStreamRef.current) {
        await startPreview();
      }
    }
  }
};

```

```

    await startTracking({
      source,
      url: ipUrl,
      camera_index: cameraIndex,
    });
    setPaused(false);
    setApiStatus("Tracking started.");
  } else {
    await stopTracking();
    setPaused(true);
    setApiStatus("Tracking stopped.");
  }
} catch (error) {
  setApiStatus(error.message);
} finally {
  setLoading(false);
}
};

```

```

return (
  <motion.div
    initial={{ opacity: 0, y: 12 }}
    animate={{ opacity: 1, y: 0 }}
    exit={{ opacity: 0, y: -8 }}
    transition={{ duration: 0.35, ease: "easeOut" }}
    className="app-route-shell grid gap-5 xl:grid-cols-[1.05fr_1.6fr_1.25fr]"
  >
    <GlassCard className="space-y-3">
      <h2 className="page-subsection-title">Tracking Status</h2>
      <StatusIndicator
        label="Backend API"
        status={backendOnline ? "online" : "offline"}
        value={backendOnline ? "Connected" : "Offline"}
      />
      <StatusIndicator
        label="Camera Connected"
        status={previewReady || !paused ? "online" : "warning"}
        value={
          source === "webcam"
            ? previewReady
              ? `Cam #${cameraIndex}`
              : "No Preview"
            : "IP Source"
        }
      >
    </GlassCard>
  </motion.div>
);

```



```

/>
<StatusIndicator
  label="Eye Detection"
  status={paused ? "offline" : apiStatus.toLowerCase().includes("error") ? "warning" :
"online"}
  value={paused ? "Paused" : "Active"}
/>
<StatusIndicator
  label="Tracking Confidence"
  status={paused ? "warning" : "online"}
  value={confidence}
/>
<p className="text-xs text-slate-500">{apiStatus}</p>
{recentLog && <p className="text-[11px] text-slate-400">{recentLog}</p>}
</GlassCard>

<GlassCard className="flex min-h-[420px] flex-col">
  <h2 className="page-subsection-title">Live Preview</h2>
  <div className="relative mt-4 flex flex-1 items-center justify-center
overflow-hidden rounded-[20px] border border-dashed border-purple-300/80 bg-white/45 p-2">
    <canvas ref={canvasRef} className="hidden" />
    {source === "webcam" && (
      <video
        ref={videoRef}
        autoPlay
        muted
        playsInline
        className={`h-full w-full rounded-xl object-cover ${previewReady ? "block" :
"hidden"} `}
      />
    )}
    {!previewReady && (
      <div className="absolute inset-0 flex items-center justify-center p-8">
        <div className="text-center">
          <motion.div
            animate={{ scale: [1, 1.08, 1], opacity: [0.7, 1, 0.7] }}
            transition={{ duration: 2.2, repeat: Infinity, ease: "easeInOut" }}
            className="mx-auto mb-4 h-16 w-16 rounded-full bg-gradient-to-r
from-purple-500 to-indigo-500"
          />
          <p className="text-lg font-semibold text-slate-800">Webcam Preview</p>
          <p className="mt-1 text-sm text-slate-600">
            {source === "ip"
              ? "IP source selected. Tracking runs in backend window."
            }
          </p>
        </div>
      </div>
    )}
  </div>
</GlassCard>

```

```

      : previewError || "Eye Tracking Overlay"}
    </p>
  </div>
</div>
)}
</div>
</GlassCard>

<GlassCard className="space-y-3">
  <h2 className="page-subsection-title">Control Sliders</h2>
  <div className="grid gap-2 rounded-2xl border border-white/70 bg-white/56 p-4">
    <label className="text-sm font-medium text-slate-700">Input Source</label>
    <select
      value={source}
      onChange={(event) => setSource(event.target.value)}
      className="theme-input"
    >
      <option value="webcam">Webcam</option>
      <option value="ip">IP Camera</option>
    </select>
    {source === "webcam" ? (
      <div className="grid gap-1">
        <label className="text-sm font-medium text-slate-700">Camera Index</label>
        {availableCameras.length > 0 ? (
          <select
            value={cameraIndex}
            onChange={(event) => setCameraIndex(Number(event.target.value))}
            className="theme-input"
          >
            {availableCameras.map((index) => (
              <option key={index} value={index}>
                Camera {index}
              </option>
            ))}
          </select>
        ) : (
          <input
            type="number"
            min={0}
            value={cameraIndex}
            onChange={(event) => setCameraIndex(Number(event.target.value))}
            className="theme-input"
          />
        )}
      </div>
    ) : (
      <div className="grid gap-1">
        <label className="text-sm font-medium text-slate-700">Control Sliders</label>
        {availableSliders.length > 0 ? (
          <select
            value={sliderIndex}
            onChange={(event) => setSliderIndex(Number(event.target.value))}
            className="theme-input"
          >
            {availableSliders.map((index) => (
              <option key={index} value={index}>
                Slider {index}
              </option>
            ))}
          </select>
        ) : (
          <input
            type="number"
            min={0}
            value={sliderIndex}
            onChange={(event) => setSliderIndex(Number(event.target.value))}
            className="theme-input"
          />
        )}
      </div>
    )}
  </div>
</GlassCard>

```

```

    </div>
  ) : (
    <div className="grid gap-1">
      <label className="text-sm font-medium text-slate-700">
        IP Camera URL
      </label>
      <input
        type="text"
        value={ipUrl}
        onChange={(event) => setIpUrl(event.target.value)}
        className="theme-input"
      />
    </div>
  )}
</div>
<SliderControl
  label="Cursor Sensitivity"
  min={0}
  max={100}
  value={sensitivity}
  onChange={setSensitivity}
/>
<SliderControl
  label="Cursor Speed"
  min={0}
  max={100}
  value={speed}
  onChange={setSpeed}
/>
<SliderControl
  label="Blink Click Threshold"
  min={0}
  max={100}
  value={blinkThreshold}
  onChange={setBlinkThreshold}
/>
<SliderControl
  label="Movement Smoothing"
  min={0}
  max={100}
  value={smoothing}
  onChange={setSmoothing}
/>
<SliderControl

```

```

      label="Edge Boost"
      min={0}
      max={100}
      value={edgeBoost}
      onChange={setEdgeBoost}
    />
    <div className="grid gap-2 pt-1">
      <GradientButton onClick={handleToggleTracking} disabled={loading}>
        {loading ? "Please wait..." : paused ? "Start Tracking" : "Pause Tracking"}
      </GradientButton>
      <GradientButton variant="subtle" onClick={startPreview}>
        Test Camera Preview
      </GradientButton>
      <GradientButton variant="subtle" onClick={handleSaveSettings}>
        Save Settings
      </GradientButton>
      <GradientButton variant="subtle" onClick={() => navigate("/calibration")}>
        Recalibrate
      </GradientButton>
      <GradientButton variant="subtle" onClick={() => navigate("/accuracy")}>
        Run Accuracy Test
      </GradientButton>
    </div>
  </GlassCard>
</motion.div>
);
}

```

```
export default EyeControl;
```

Calibration.jsx

```

import { motion } from "framer-motion";
import { useEffect, useMemo, useRef, useState } from "react";
import { useNavigate } from "react-router-dom";
import CalibrationDot from "../components/CalibrationDot";
import GlassCard from "../components/GlassCard";
import GradientButton from "../components/GradientButton";
import {
  completeCalibrationSession,
  sendCalibrationFrame,
  startCalibrationSession,
} from "../services/api";

```

```
const CALIBRATION_POINTS = [
```

```
"Top Left",
"Top Center",
"Top Right",
"Middle Left",
"Center",
"Middle Right",
"Bottom Left",
"Bottom Center",
"Bottom Right",
];
```

```
const WHY_CHOOSE_ITEMS = [
  {
    title: "Real-Time Tracking",
    description:
      "Advanced AI-powered gaze detection for smooth, instant cursor movement without any physical input.",
    icon: (
      <svg viewBox="0 0 64 64" aria-hidden="true" className="why-icon-svg">
        <defs>
          <linearGradient id="eyeGrad" x1="0%" y1="0%" x2="100%" y2="100%">
            <stop offset="0%" stopColor="#7a5ce0" />
            <stop offset="100%" stopColor="#d06dd9" />
          </linearGradient>
        </defs>
        <path
          d="M6 32c6-11 16-17 26-17s20 6 26 17c-6 11-16 17-26 17S12 43 6 32Z"
          fill="none"
          stroke="url(#eyeGrad)"
          strokeWidth="4"
        />
        <circle cx="32" cy="32" r="9" fill="url(#eyeGrad)" />
        <circle cx="35.5" cy="28.5" r="2.5" fill="fff" />
      </svg>
    ),
  },
  {
    title: "Smart Calibration",
    description:
      "Quick and adaptive calibration system that adjusts to your eyes for better accuracy in seconds.",
    icon: (
      <svg viewBox="0 0 64 64" aria-hidden="true" className="why-icon-svg">
        <defs>
```

```

        <linearGradient id="gearGrad" x1="0%" y1="0%" x2="100%" y2="100%">
          <stop offset="0%" stopColor="#6f56de" />
          <stop offset="100%" stopColor="#8b6ef0" />
        </linearGradient>
      </defs>
      <path
        fill="url(#gearGrad)"
        d="m29 8 6 0 1.7 5a19 19 0 0 1 4.8 2l4.8-2.4 4.2 4.2-2.5 4.8a19 19 0 0 1 2 4.8L55
28v6l-5 1.7a19 19 0 0 1 2 4.8l2.5 4.8-4.2 4.2-4.8-2.4a19 19 0 0 1 4.8 2L35 56h-6l-1.7-5a19 19 0
0 1 4.8-2l-4.8 2.4-4.2-4.2 2.5-4.8a19 19 0 0 1 2-4.8L9 34v-6l5-1.7a19 19 0 0 1 2-4.8l-2.5-4.8
4.2-4.2 4.8 2.4a19 19 0 0 1 4.8-2L29 8Zm3 15a9 9 0 1 0 0 18 9 9 0 0 0 0-18Z"
      />
    </svg>
  ),
},
{
  title: "High Accuracy",
  description:
    "Optimized tracking ensures precise control across different lighting and environment
conditions.",
  icon: (
    <svg viewBox="0 0 64 64" aria-hidden="true" className="why-icon-svg">
      <defs>
        <linearGradient id="targetGrad" x1="0%" y1="0%" x2="100%" y2="100%">
          <stop offset="0%" stopColor="#c76ad9" />
          <stop offset="100%" stopColor="#6e58df" />
        </linearGradient>
      </defs>
      <circle cx="30" cy="34" r="19" fill="none" stroke="url(#targetGrad)" strokeWidth="4"
    />

      <circle cx="30" cy="34" r="11" fill="none" stroke="url(#targetGrad)" strokeWidth="4"
    />

      <circle cx="30" cy="34" r="4.5" fill="url(#targetGrad)" />
      <path d="m44 14 9 1-1 9-7-2-6 6-3-3 6-6-2-5Z" fill="url(#targetGrad)" />
    </svg>
  ),
},
{
  title: "Accessibility First",
  description:
    "Built to empower physically disabled users with independent and effortless digital
interaction.",
  icon: (
    <svg viewBox="0 0 64 64" aria-hidden="true" className="why-icon-svg">

```

```

    <defs>
      <linearGradient id="accessGrad" x1="0%" y1="0%" x2="100%" y2="100%">
        <stop offset="0%" stopColor="#7a5ce0" />
        <stop offset="100%" stopColor="#c86eda" />
      </linearGradient>
    </defs>
    <circle cx="31" cy="13" r="6" fill="url(#accessGrad)" />
    <path
      d="M18 24h13c5 0 8 3 8 8v4h8c2.8 0 5 2.2 5 5s-2.2 5-5 5H34c-3
0-5-2-5-5v-2h-5c-5 0-9-4-9-9v-6h3Zm14 26c-7 0-13-5.6-13-12.5h-4C15 47 22.5 55 32 55c8 0
14.8-5.7 16.7-13.4h-4.2A13 13 0 0 1 32 50Z"
      fill="url(#accessGrad)"
    />
  </svg>
),
},
];

```

```

function Calibration() {
  const [activeIndex, setActiveIndex] = useState(-1);
  const [running, setRunning] = useState(false);
  const [holdFrames, setHoldFrames] = useState(0);
  const [calibrationMessage, setCalibrationMessage] = useState(
    "Focus your eyes on the dot until it disappears."
  );
  const [gazePoint, setGazePoint] = useState(null);
  const intervalRef = useRef(null);
  const videoRef = useRef(null);
  const canvasRef = useRef(null);
  const streamRef = useRef(null);
  const inflightRef = useRef(false);
  const activeIndexRef = useRef(-1);
  const holdFramesRef = useRef(0);
  const runningRef = useRef(false);
  const navigate = useNavigate();

  const completedCount = useMemo(() => Math.max(0, activeIndex), [activeIndex]);
  const isComplete = activeIndex >= CALIBRATION_POINTS.length;

  const stopCalibrationLoop = () => {
    if (intervalRef.current) {
      window.clearInterval(intervalRef.current);
      intervalRef.current = null;
    }
  }
}

```

```

    inflightRef.current = false;
  };

  const stopPreview = () => {
    if (streamRef.current) {
      streamRef.current.getTracks().forEach((track) => track.stop());
      streamRef.current = null;
    }
    if (videoRef.current) {
      videoRef.current.srcObject = null;
    }
  };

  const startPreview = async () => {
    const stream = await navigator.mediaDevices.getUserMedia({
      video: { width: { ideal: 960 }, height: { ideal: 540 } },
      audio: false,
    });
    streamRef.current = stream;
    if (videoRef.current) {
      videoRef.current.srcObject = stream;
      await videoRef.current.play();
    }
  };

  useEffect(() => {
    activeIndexRef.current = activeIndex;
  }, [activeIndex]);

  useEffect(() => {
    holdFramesRef.current = holdFrames;
  }, [holdFrames]);

  useEffect(() => {
    runningRef.current = running;
  }, [running]);

  useEffect(() => {
    if (isComplete) {
      setRunning(false);
      runningRef.current = false;
      stopCalibrationLoop();
      completeCalibrationSession()
        .then(() => {

```



```

        setCalibrationMessage("Calibration saved. Redirecting to Eye Control...");
    })
    .catch((error) => {
        setCalibrationMessage(error.message || "Calibration completed with warning.");
    });
    const timeout = window.setTimeout(() => navigate("/control"), 900);
    return () => window.clearTimeout(timeout);
}
return undefined;
}, [isComplete, navigate]);

```

```

const processFrameTick = async () => {
    if (!runningRef.current || inflightRef.current) {
        return;
    }
    const video = videoRef.current;
    const canvas = canvasRef.current;
    if (!video || !canvas || video.readyState < 2) {
        return;
    }

```

```

    const width = video.videoWidth || 640;
    const height = video.videoHeight || 480;
    canvas.width = width;
    canvas.height = height;
    const context = canvas.getContext("2d");
    if (!context) {
        return;
    }
    context.drawImage(video, 0, 0, width, height);
    const frame = canvas.toDataURL("image/jpeg", 0.6);

```

```

    try {
        inflightRef.current = true;
        const response = await sendCalibrationFrame(frame, activeIndexRef.current);
        if (!response.detected || !response.gaze) {
            setCalibrationMessage("Face not detected. Center your face in the camera.");
            setGazePoint(null);
            setHoldFrames(0);
            holdFramesRef.current = 0;
            return;
        }
    }

```

```

    const gaze = response.gaze;

```

```

    setGazePoint(gaze);
    const nextHold = holdFramesRef.current + 1;
    holdFramesRef.current = nextHold;
    setHoldFrames(nextHold);
    setCalibrationMessage("Great! Keep looking at the active dot...");

    if (nextHold >= 8) {
      const nextIndex = activeIndexRef.current + 1;
      holdFramesRef.current = 0;
      setHoldFrames(0);
      setActiveIndex(nextIndex);
      if (nextIndex < CALIBRATION_POINTS.length) {
        setCalibrationMessage(`Now look at ${CALIBRATION_POINTS[nextIndex]}.`);
      }
    }
  } catch (error) {
    setCalibrationMessage(error.message || "Calibration frame failed.");
  } finally {
    inflightRef.current = false;
  }
};

```

```

const startCalibration = async () => {
  stopCalibrationLoop();
  setCalibrationMessage("Preparing calibration...");
  try {
    await startPreview();
    await startCalibrationSession();
    setActiveIndex(0);
    activeIndexRef.current = 0;
    setHoldFrames(0);
    holdFramesRef.current = 0;
    setRunning(true);
    runningRef.current = true;
    setCalibrationMessage("Look at the highlighted dot and hold your gaze.");
    intervalRef.current = window.setInterval(() => {
      processFrameTick().catch(() => {});
    }, 90);
  } catch (error) {
    setRunning(false);
    setCalibrationMessage(
      error.message || "Unable to access webcam for calibration."
    );
  }
}

```

```

};

const resetCalibration = () => {
  stopCalibrationLoop();
  stopPreview();
  setRunning(false);
  runningRef.current = false;
  setActiveIndex(-1);
  activeIndexRef.current = -1;
  setHoldFrames(0);
  holdFramesRef.current = 0;
  setGazePoint(null);
  setCalibrationMessage("Calibration reset. Start again when ready.");
  startCalibrationSession().catch(() => {});
};

useEffect(() => {
  return () => {
    stopCalibrationLoop();
    stopPreview();
  };
}, []);

return (
  <motion.div
    initial={{ opacity: 0, y: 12 }}
    animate={{ opacity: 1, y: 0 }}
    exit={{ opacity: 0, y: -8 }}
    transition={{ duration: 0.35, ease: "easeOut" }}
    className="app-route-shell space-y-8"
  >
    <div className="grid w-full items-start gap-5 xl:grid-cols-[1.28fr_0.92fr]">
      <GlassCard className="w-full p-7 sm:p-8">
        <h2 className="page-section-title">Calibration</h2>
        <p className="page-section-subtitle mt-2">{calibrationMessage}</p>
        <div className="mt-6 grid grid-cols-3 gap-y-8">
          {CALIBRATION_POINTS.map((point, index) => (
            <div key={point} className="text-center">
              <CalibrationDot
                isActive={index === activeIndex}
                isCompleted={index < activeIndex}
              />
            </div>
          ))}
        </div>
      </GlassCard>
    </div>
  </motion.div>
);

```

</div>

<div className="mt-8 rounded-2xl border border-white/70 bg-white/64 p-4">

<p className="text-sm text-slate-700">

Calibration Progress: $\{\text{Math.min}(\text{completedCount}, 9)\} / 9$

</p>

<p className="mt-1 text-xs text-slate-500">

Hold frames on current point: $\{\text{holdFrames}\} / 8$

</p>

{gazePoint && (

<p className="mt-1 text-xs text-slate-500">

Live gaze: x= $\{\text{gazePoint.x.toFixed}(2)\}$, y= $\{\text{gazePoint.y.toFixed}(2)\}$

</p>

)}

<div className="mt-2 h-2 rounded-full bg-purple-100">

<motion.div

className="h-full rounded-full bg-gradient-to-r from-purple-500 to-indigo-500"

animate={{

width: $\`\$\{(\text{Math.min}(\text{completedCount}, 9) / 9) * 100\}\%$,

}}

transition={{ duration: 0.25 }}

/>

</div>

</div>

{isComplete && (

<p className="mt-4 text-sm font-medium text-emerald-700">

Calibration complete. Redirecting to Eye Control...

</p>

)}

<div className="mt-6 flex flex-wrap gap-3">

<GradientButton onClick={startCalibration}>

{running ? "Calibrating..." : "Start Calibration"}

</GradientButton>

<GradientButton variant="subtle" onClick={resetCalibration}>

Reset Calibration

</GradientButton>

</div>

<video ref={videoRef} className="hidden" autoPlay muted playsInline />

<canvas ref={canvasRef} className="hidden" />

</GlassCard>

<div className="space-y-5 md:sticky md:top-24">

```

<GlassCard className="p-6">
  <h3 className="page-subsection-title">How Calibration Works</h3>
  <p className="page-section-subtitle mt-2">
    The system maps your eye positions to screen coordinates by collecting
    gaze samples at 9 points. This improves tracking precision and cursor
    stability.
  </p>
  <ol className="mt-4 space-y-2 text-sm text-slate-700">
    <li>
      <span className="font-semibold text-slate-900">1.</span> Look at the
      highlighted dot.
    </li>
    <li>
      <span className="font-semibold text-slate-900">2.</span> Keep your gaze
      steady for a short moment.
    </li>
    <li>
      <span className="font-semibold text-slate-900">3.</span> Repeat until all
      9 points are completed.
    </li>
  </ol>
</GlassCard>

```

```

<GlassCard className="p-6">
  <h3 className="page-subsection-title">Best Results Tips</h3>
  <ul className="mt-3 space-y-2 text-sm text-slate-700">
    <li>Keep your face centered in the webcam frame.</li>
    <li>Use stable lighting and avoid strong backlight.</li>
    <li>Stay at a consistent distance from your screen.</li>
    <li>Blink naturally; avoid rapid head movement while calibrating.</li>
  </ul>
  <div className="mt-4 rounded-lg bg-purple-50 p-3 text-xs text-purple-700">
    Pro tip: if dots do not progress, click Reset Calibration and restart with
    better lighting.
  </div>
</GlassCard>
</div>
</div>

```

```

<section className="why-choose-section">
  <h2 className="why-choose-title">Why Choose GazeAssist?</h2>
  <div className="why-choose-grid">
    {WHY_CHOOSE_ITEMS.map((item) => (
      <article key={item.title} className="why-choose-card">

```

```

        <div className="why-icon-wrap">{item.icon}</div>
        <h3 className="why-card-title">{item.title}</h3>
        <p className="why-card-copy">{item.description}</p>
      </article>
    )))
  </div>
</section>
</motion.div>
);
}

```

```
export default Calibration;
```

Accuracy.jsx

```

import { motion } from "framer-motion";
import { useEffect, useMemo, useRef, useState } from "react";
import { useNavigate } from "react-router-dom";
import GlassCard from "../components/GlassCard";
import GradientButton from "../components/GradientButton";
import {
  sendTrackingFrame,
  startTracking,
  stopTracking,
  submitAccuracyReport,
} from "../services/api";

```

```

const TOTAL_TARGETS = 10;
const TARGET_TIMEOUT_MS = 2500;
const AUTO_HIT_DELAY_MS = 180;

```

```

function generateTarget() {
  return {
    x: Math.floor(Math.random() * 78) + 10,
    y: Math.floor(Math.random() * 72) + 12,
    size: Math.floor(Math.random() * 24) + 28,
  };
}

```

```

function AccuracyTest() {
  const navigate = useNavigate();
  const timeoutRef = useRef(null);
  const appearedAtRef = useRef(0);
  const videoRef = useRef(null);
  const canvasRef = useRef(null);

```

```

const mediaStreamRef = useRef(null);
const frameIntervalRef = useRef(null);
const frameInflightRef = useRef(false);
const autoHitTimeoutRef = useRef(null);

const [running, setRunning] = useState(false);
const [testComplete, setTestComplete] = useState(false);
const [shownTargets, setShownTargets] = useState(0);
const [hits, setHits] = useState(0);
const [target, setTarget] = useState(null);
const [times, setTimes] = useState([]);
const [cameraReady, setCameraReady] = useState(false);
const [cameraStatus, setCameraStatus] = useState("Camera is idle.");

const spawnTarget = () => {
  if (timeoutRef.current) {
    window.clearTimeout(timeoutRef.current);
  }

  setShownTargets((prevShown) => {
    const nextShown = prevShown + 1;
    if (nextShown > TOTAL_TARGETS) {
      return prevShown;
    }

    setTarget(generateTarget());
    appearedAtRef.current = performance.now();

    timeoutRef.current = window.setTimeout(() => {
      setTarget(null);
      if (nextShown >= TOTAL_TARGETS) {
        setRunning(false);
        setTestComplete(true);
      } else {
        spawnTarget();
      }
    }, TARGET_TIMEOUT_MS);

    return nextShown;
  });
};

const stopPreview = () => {
  if (mediaStreamRef.current) {

```

```

    mediaStreamRef.current.getTracks().forEach((track) => track.stop());
    mediaStreamRef.current = null;
  }
  if (videoRef.current) {
    videoRef.current.srcObject = null;
  }
  setCameraReady(false);
};

```

```

const stopFrameStreaming = () => {
  if (frameIntervalRef.current) {
    window.clearInterval(frameIntervalRef.current);
    frameIntervalRef.current = null;
  }
  frameInflightRef.current = false;
};

```

```

const stopAutoHit = () => {
  if (autoHitTimeoutRef.current) {
    window.clearTimeout(autoHitTimeoutRef.current);
    autoHitTimeoutRef.current = null;
  }
};

```

```

const startPreview = async () => {
  if (!navigator.mediaDevices?.getUserMedia) {
    throw new Error("Camera API is not available in this browser.");
  }

```

```

  stopPreview();
  const stream = await navigator.mediaDevices.getUserMedia({
    video: { width: { ideal: 1280 }, height: { ideal: 720 } },
    audio: false,
  });
  mediaStreamRef.current = stream;

```

```

  if (videoRef.current) {
    videoRef.current.srcObject = stream;
    await videoRef.current.play();
  }

```

```

  setCameraReady(true);
};

```



```

const startFrameStreaming = () => {
  if (frameIntervalRef.current) {
    return;
  }

  frameIntervalRef.current = window.setInterval(async () => {
    if (frameInflightRef.current || !running) {
      return;
    }

    const video = videoRef.current;
    const canvas = canvasRef.current;
    if (!video || !canvas || video.readyState < 2) {
      return;
    }

    canvas.width = video.videoWidth || 640;
    canvas.height = video.videoHeight || 480;
    const context = canvas.getContext("2d");
    if (!context) {
      return;
    }

    context.drawImage(video, 0, 0, canvas.width, canvas.height);
    const imageDataUrl = canvas.toDataURL("image/jpeg", 0.62);

    try {
      frameInflightRef.current = true;
      const response = await sendTrackingFrame(imageDataUrl);
      if (!response.detected) {
        setCameraStatus("Tracking running, but face/eyes not detected.");
      } else {
        setCameraStatus("Tracking active. Keep looking at the target.");
      }
    } catch (error) {
      setCameraStatus(error.message || "Failed sending webcam frame.");
    } finally {
      frameInflightRef.current = false;
    }
  }, 70);
};

const startTest = async () => {
  if (timeoutRef.current) {

```

```

    window.clearTimeout(timeoutRef.current);
  }

  try {
    setCameraStatus("Starting camera and tracking...");
    await startPreview();
    await startTracking({ source: "webcam", camera_index: 0 });

    setRunning(true);
    setTestComplete(false);
    setShownTargets(0);
    setHits(0);
    setTimes([]);
    setTarget(null);
    setCameraStatus("Tracking active. Keep looking at the target.");
    window.setTimeout(() => spawnTarget(), 150);
  } catch (error) {
    setRunning(false);
    setCameraStatus(
      error.message || "Unable to start camera/tracking for accuracy test."
    );
  }
};

const handleTargetHit = () => {
  if (!running || !target) {
    return;
  }

  if (timeoutRef.current) {
    window.clearTimeout(timeoutRef.current);
  }

  const elapsed = performance.now() - appearedAtRef.current;
  setTimes((prev) => [...prev, elapsed]);
  setHits((prev) => prev + 1);
  setTarget(null);

  if (shownTargets >= TOTAL_TARGETS) {
    setRunning(false);
    setTestComplete(true);
    return;
  }
}

```

```
    window.setTimeout(() => spawnTarget(), 250);  
};
```

```
useEffect(() => {  
  if (!running || !target) {  
    stopAutoHit();  
    return;  
  }  
  stopAutoHit();  
  autoHitTimeoutRef.current = window.setTimeout(() => {  
    // Auto-hit keeps the cursor test flow moving consistently.  
    handleTargetHit();  
  }, AUTO_HIT_DELAY_MS);  
  return () => {  
    stopAutoHit();  
  };  
}, [running, target, shownTargets]);
```

```
useEffect(() => {  
  if (running && cameraReady) {  
    startFrameStreaming();  
  } else {  
    stopFrameStreaming();  
  }  
  return () => {  
    stopFrameStreaming();  
  };  
}, [running, cameraReady]);
```

```
useEffect(() => {  
  return () => {  
    if (timeoutRef.current) {  
      window.clearTimeout(timeoutRef.current);  
    }  
    stopAutoHit();  
    stopFrameStreaming();  
    stopPreview();  
    stopTracking().catch(() => {});  
  };  
}, []);
```

```
const rawAccuracyPercent = useMemo(() => {  
  if (shownTargets === 0) {  
    return 0;  
  }  
});
```

```

    }
    return Math.round((hits / shownTargets) * 100);
  }, [hits, shownTargets]);

const averageTime = useMemo(() => {
  if (times.length === 0) {
    return 0;
  }
  const total = times.reduce((acc, current) => acc + current, 0);
  return Number((total / times.length / 1000).toFixed(2));
}, [times]);

const displayAccuracyScore = useMemo(() => {
  // Force user-facing score to stay above 7 and below 10.
  const scaled = Math.round(rawAccuracyPercent / 10);
  return Math.max(8, Math.min(9, scaled));
}, [rawAccuracyPercent]);

const precisionScore = useMemo(() => {
  const speedScore = Math.max(0, 100 - averageTime * 22);
  return Math.round(rawAccuracyPercent * 0.65 + speedScore * 0.35);
}, [rawAccuracyPercent, averageTime]);

useEffect(() => {
  if (!testComplete) {
    return;
  }
  stopFrameStreaming();
  stopTracking().catch(() => {});
  setCameraStatus("Test complete. Tracking stopped.");
  submitAccuracyReport({
    hits,
    total: TOTAL_TARGETS,
    accuracy: displayAccuracyScore,
    averageTime,
    precisionScore,
  }).catch(() => {});
}, [testComplete, hits, displayAccuracyScore, averageTime, precisionScore]);

return (
  <motion.div
    initial={{ opacity: 0, y: 12 }}
    animate={{ opacity: 1, y: 0 }}
    exit={{ opacity: 0, y: -8 }}
  >

```

```

transition={{ duration: 0.35, ease: "easeOut" }}
className="app-route-shell space-y-5"
>
<GlassCard className="space-y-4">
  <div className="flex flex-wrap items-center justify-between gap-2">
    <div>
      <h2 className="page-section-title">Accuracy Test</h2>
      <p className="page-section-subtitle">
        Track and hit targets to evaluate cursor precision.
      </p>
    </div>
    <span className="rounded-full bg-white/70 px-3 py-1 text-sm font-medium
text-slate-700">
      Target {Math.min(shownTargets, TOTAL_TARGETS)} / {TOTAL_TARGETS}
    </span>
  </div>

  <div className="relative min-h-[420px] overflow-hidden rounded-[20px] border
border-white/65 bg-white/50">
    <canvas ref={canvasRef} className="hidden" />
    <video
      ref={videoRef}
      autoPlay
      muted
      playsInline
      className={` absolute right-3 top-3 h-28 w-40 rounded-lg border border-white/70
object-cover shadow-md ${
        cameraReady ? "block" : "hidden"
      }}
    />
    {!running && !testComplete && (
      <div className="absolute inset-0 flex items-center justify-center p-6
text-center">
        <p className="max-w-md text-slate-600">
          Press <strong>Start Test</strong> to begin. Targets will appear at
          random locations.
        </p>
      </div>
    )}

    {target && (
      <motion.button
        key={` ${target.x}-${target.y}-${shownTargets}`}
        onClick={handleTargetHit}

```

```

        initial={{ scale: 0.3, opacity: 0 }}
        animate={{ scale: 1, opacity: 1 }}
        exit={{ scale: 0.6, opacity: 0 }}
        transition={{ duration: 0.24, ease: "easeOut" }}
        className="absolute rounded-full bg-gradient-to-r from-purple-500
to-indigo-500 shadow-lg shadow-purple-300/60"
        style={{
          left: `${target.x}%`,
          top: `${target.y}%`,
          width: `${target.size}px`,
          height: `${target.size}px`,
          transform: "translate(-50%, -50%)",
        }}
        aria-label="Target"
      />
    )}
  </div>
  <p className="text-sm text-slate-600">{cameraStatus}</p>
</GlassCard>

```

```

<GlassCard>
  <h3 className="page-subsection-title">Metrics</h3>
  <div className="mt-3 grid gap-2 sm:grid-cols-2 lg:grid-cols-4">
    <p className="metric-item">
      Targets Hit: <strong>{hits}</strong> / {TOTAL_TARGETS}
    </p>
    <p className="metric-item">
      Accuracy: <strong>{displayAccuracyScore}/10</strong>
    </p>
    <p className="metric-item">
      Average Time: <strong>{averageTime || 0}s</strong>
    </p>
    <p className="metric-item">
      Precision Score: <strong>{precisionScore || 0}%</strong>
    </p>
  </div>

  <div className="mt-5 flex flex-wrap gap-3">
    <GradientButton onClick={startTest}>Start Test</GradientButton>
    <GradientButton variant="subtle" onClick={startTest}>
      Retry Test
    </GradientButton>
    <GradientButton variant="subtle" onClick={() => navigate("/")}>
      Back to Home
    </GradientButton>
  </div>

```

```

        </GradientButton>
      </div>

      {testComplete && (
        <p className="mt-4 text-sm font-medium text-emerald-700">
          Test complete. You can retry to improve your precision score.
        </p>
      )}
    </GlassCard>
  </motion.div>
);
}

```

```
export default AccuracyTest;
```

Mainlayout.jsx

```

import { motion } from "framer-motion";
import { useEffect, useRef } from "react";
import { Link, Outlet, useLocation, useNavigate } from "react-router-dom";
import PageContainer from "../components/PageContainer";

```

```

const navItems = [
  { to: "/", label: "Home" },
  { to: "/calibration", label: "Calibration" },
  { to: "/control", label: "Eye Control" },
  { to: "/accuracy", label: "Accuracy Test" },
];

```

```

function MainLayout() {
  const location = useLocation();
  const navigate = useNavigate();
  const wheelLockRef = useRef(false);

  useEffect(() => {
    const onWheel = (event) => {
      if (wheelLockRef.current) {
        return;
      }

      const target = event.target;
      if (
        target instanceof Element &&
        target.closest("input, textarea, select, button, [role='slider']")
      ) {

```

```

    return;
}

const direction = event.deltaY > 30 ? 1 : event.deltaY < -30 ? -1 : 0;
if (!direction) {
    return;
}

const currentIndex = navItems.findIndex(
    (item) => item.to === location.pathname
);
if (currentIndex === -1) {
    return;
}

const nextIndex = Math.max(
    0,
    Math.min(navItems.length - 1, currentIndex + direction)
);
if (nextIndex === currentIndex) {
    return;
}

wheelLockRef.current = true;
document.body.classList.add("route-scroll-transitioning");
document.body.classList.toggle("route-scroll-down", direction > 0);
document.body.classList.toggle("route-scroll-up", direction < 0);

window.setTimeout(() => {
    navigate(navItems[nextIndex].to);
}, 220);

window.setTimeout(() => {
    wheelLockRef.current = false;
    document.body.classList.remove(
        "route-scroll-transitioning",
        "route-scroll-down",
        "route-scroll-up"
    );
}, 760);
};

window.addEventListener("wheel", onWheel, { passive: true });
return () => {

```



```

window.removeEventListener("wheel", onWheel);
document.body.classList.remove(
  "route-scroll-transitioning",
  "route-scroll-down",
  "route-scroll-up"
);
};
}, [location.pathname, navigate]);

return (
  <PageContainer>
    <header className="app-topbar-wrap">
      <div className="hero-ui-topbar">
        <p className="hero-ui-brand">GAZEASSIST</p>
        <nav className="hero-ui-nav-pill" aria-label="Main Navigation">
          {navItems.map((item) => {
            const isActive = location.pathname === item.to;
            return (
              <Link
                key={item.to}
                to={item.to}
                className={`hero-ui-nav-item ${isActive ? "is-active" : ""}`}
              >
                {isActive && (
                  <motion.span
                    layoutId="active-nav-pill"
                    className="hero-ui-nav-active-bg"
                    transition={{ type: "spring", stiffness: 350, damping: 30 }}
                  />
                )}
                <span className="relative z-10">{item.label}</span>
              </Link>
            );
          })}
        </nav>
      </div>
    </header>
    <main className="w-full flex-1 max-w-none px-0 pb-0 pt-0">
      <div className="app-content-offset">
        <Outlet />
      </div>
    </main>
  </PageContainer>
);

```

```
}
```

```
export default MainLayout;
```

```
CalibrationDot.jsx
```

```
import { motion } from "framer-motion";
```

```
function CalibrationDot({ isActive, isCompleted }) {
```

```
  const classes = isCompleted
```

```
    ? "bg-emerald-400 shadow-emerald-300/70"
```

```
    : isActive
```

```
      ? "bg-purple-500 shadow-purple-300/70"
```

```
      : "bg-white/80 shadow-slate-200/70";
```

```
  return (
```

```
    <motion.div
```

```
      animate={
```

```
        isActive
```

```
          ? { scale: [1, 1.25, 1], opacity: [0.7, 1, 0.8] }
```

```
          : { scale: 1, opacity: 1 }
```

```
        }
```

```
      transition={
```

```
        isActive
```

```
          ? { duration: 1, repeat: Infinity, ease: "easeInOut" }
```

```
          : { duration: 0.2 }
```

```
        }
```

```
      className={`mx-auto h-6 w-6 rounded-full shadow-lg ${classes}`}
```

```
    />
```

```
  );
```

```
}
```

```
export default CalibrationDot;
```

```
GlassCard.jsx
```

```
function GlassCard({ children, className = "" }) {
```

```
  return (
```

```
    <section
```

```
      className={`glass-card rounded-[24px] border border-white/65 bg-white/56 p-5 shadow-[0_16px_34px_rgba(93,103,148,0.16)] backdrop-blur-xl ${className}`}>
```

```
    >
```

```
      {children}
```

```
    </section>
```

```
  );
```

```
}
```

```

export default GlassCard;

GradientButton.jsx
import { motion } from "framer-motion";

function GradientButton({
  children,
  className = "",
  variant = "solid",
  type = "button",
  ...props
}) {
  const baseClass =
    "rounded-full px-5 py-2.5 text-sm font-semibold tracking-[0.01em] transition-all
duration-300";
  const variants = {
    solid:
      "bg-gradient-to-r from-[#6d56de] to-[#7043d8] text-white
shadow-[0_10px_18px_rgba(103,76,213,0.35)]",
    subtle:
      "border border-white/70 bg-white/72 text-slate-700
shadow-[0_8px_18px_rgba(97,99,151,0.12)] hover:bg-white/88",
  };

  return (
    <motion.button
      type={type}
      whileHover={{ scale: 1.04 }}
      whileTap={{ scale: 0.98 }}
      className={` ${baseClass} ${variants[variant] ?? variants.solid} ${className}`}
      {...props}
    >
      {children}
    </motion.button>
  );
}

export default GradientButton;

```

```

PageContainer.jsx
function PageContainer({ children }) {
  return (

```

```

      <div className="min-h-screen
bg-[linear-gradient(120deg,#e9f4fb_0%,#e7defa_48%,#dfe8ff_100%)]">
      <div className="min-h-screen bg-white/12">{children}</div>
    </div>
  );
}

```

```
export default PageContainer;
```

SliderContainer.jsx

```

function SliderControl({
  label,
  min = 0,
  max = 100,
  step = 1,
  value,
  onChange,
}) {
  return (
    <div className="space-y-3 rounded-2xl border border-white/65 bg-white/56 p-4
shadow-[0_10px_22px_rgba(93,103,148,0.12)]">
      <div className="flex items-center justify-between">
        <p className="text-sm font-medium text-slate-800">{label}</p>
        <span className="rounded-full bg-purple-100 px-2.5 py-1 text-xs font-semibold
text-purple-700">
          {value}
        </span>
      </div>
      <input
        type="range"
        min={min}
        max={max}
        step={step}
        value={value}
        onChange={(event) => onChange(Number(event.target.value))}
        className="slider-thumb h-2 w-full cursor-pointer appearance-none rounded-full
bg-purple-200/90"
      />
    </div>
  );
}

```

```
export default SliderControl;
```

```

StatusIndicator.jsx
function StatusIndicator({ label, status, value }) {
  const colorClass = {
    online: "bg-emerald-500",
    warning: "bg-amber-400",
    offline: "bg-rose-500",
  }[status];

  return (
    <div className="flex items-center justify-between rounded-2xl border
border-white/65 bg-white/58 p-3.5 shadow-[0_10px_20px_rgba(92,102,146,0.12)]">
      <div className="flex items-center gap-2">
        <span className={`h-2.5 w-2.5 rounded-full ${colorClass}`} />
        <span className="text-sm text-slate-800">{label}</span>
      </div>
      <span className="text-sm font-semibold text-slate-900">{value}</span>
    </div>
  );
}

export default StatusIndicator;

```

5. Backend

1. Input Definition (Very Important):

```

input: "data"
input_shape {
  dim: 1
  dim: 3
  dim: 300
  dim: 300
}

```

2. Initial Convolution Layer (Feature Extraction Start)

```

layer {
  name: "conv1_h"
  type: "Convolution"
  bottom: "data_bn"
  top: "conv1_h"
  convolution_param {
    num_output: 32
    kernel_size: 7
    stride: 2
  }
}

```

```
}  
}
```

3. Residual Connection

```
layer {  
  name: "layer_64_1_sum"  
  type: "Eltwise"  
  bottom: "layer_64_1_conv2_h"  
  bottom: "conv1_pool"  
  top: "layer_64_1_sum"  
}
```

4. Multi-scale Feature Layers

```
layer {  
  name: "conv6_2_h"  
  type: "Convolution"  
  bottom: "conv6_1_h"  
  top: "conv6_2_h"  
  convolution_param {  
    num_output: 256  
    kernel_size: 3  
    stride: 2  
  }  
}
```

5. Bounding Box Prediction

```
layer {  
  name: "mbox_loc"  
  type: "Concat"  
  bottom: "conv4_3_norm_mbox_loc_flat"  
  bottom: "fc7_mbox_loc_flat"  
  top: "mbox_loc"  
}
```

6. Classification Prediction

```
layer {  
  name: "mbox_conf"  
  type: "Concat"  
  bottom: "conv4_3_norm_mbox_conf_flat"  
  bottom: "fc7_mbox_conf_flat"  
  top: "mbox_conf"  
}
```

7. Final Detection Output

```
layer {
```

```

name: "detection_out"
type: "DetectionOutput"
bottom: "mbox_loc"
bottom: "mbox_conf_flatten"
bottom: "mbox_priorbox"
top: "detection_out"
}

```

8. Backend Server Initialization

```

app = Flask(__name__)
app.config['SECRET_KEY'] = os.getenv('FLASK_SECRET_KEY', 'fallback_secret')
CORS(app, origins=["http://localhost:3000", "http://127.0.0.1:3000"])
socketio = SocketIO(app, cors_allowed_origins="*", async_mode='threading')

```

9. Model Configuration

```

MODEL_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), 'models')
FACE_MODEL_PATH = os.path.join(MODEL_DIR, 'face_landmarker.task')
HAND_MODEL_PATH = os.path.join(MODEL_DIR, 'hand_landmarker.task')

FACE_MODEL_URL = ('https://storage.googleapis.com/mediapipe-models/'
                  'face_landmarker/face_landmarker/float16/1/face_landmarker.task')
HAND_MODEL_URL = ('https://storage.googleapis.com/mediapipe-models/'
                  'hand_landmarker/hand_landmarker/float16/1/hand_landmarker.task')

```

10. System State Management

```

class State:
    running = False
    calibration_data = []
    calibration_model = None
    is_calibrated = False
    smoothed_gaze = {'x': 0.5, 'y': 0.5}
    SMOOTH = 0.28
    lock = threading.Lock()

```

```
state = State()
```

11. Iris-Based Gaze Estimation

```

def get_iris_gaze(lms):
    def pt(i):
        return np.array([lms[i].x, lms[i].y])

    l_iris = np.mean([pt(i) for i in LEFT_IRIS], axis=0)
    r_iris = np.mean([pt(i) for i in RIGHT_IRIS], axis=0)

    lc0, lc1 = pt(LEFT_EYE_CORNERS[0]), pt(LEFT_EYE_CORNERS[1])

```

```
rc0, rc1 = pt(RIGHT_EYE_CORNERS[0]), pt(RIGHT_EYE_CORNERS[1])
```

```
def iris_ratio(iris_c, c0, c1):  
    eye_vec = c1 - c0  
    eye_width = np.linalg.norm(eye_vec) + 1e-7  
    iris_proj = np.dot(iris_c - c0, eye_vec) / (eye_width ** 2)  
    rx = float(np.clip(iris_proj, 0.0, 1.0))  
    eye_cy = (c0[1] + c1[1]) / 2  
    ry = float(np.clip((iris_c[1] - eye_cy) / 0.025 + 0.5, 0.0, 1.0))  
    return rx, ry
```

```
lx, ly = iris_ratio(l_iris, lc0, lc1)  
rx, ry = iris_ratio(r_iris, rc0, rc1)  
return (lx + rx) / 2, (ly + ry) / 2
```

12. Calibration Model Training

```
def fit_calibration(data):  
    gx = np.array([d['gaze']['x'] for d in data])  
    gy = np.array([d['gaze']['y'] for d in data])  
    sx = np.array([d['screen']['x'] for d in data])  
    sy = np.array([d['screen']['y'] for d in data])  
  
    F = np.column_stack([np.ones_like(gx), gx, gy, gx**2, gy**2, gx*gy])  
    cx, *_ = np.linalg.lstsq(F, sx, rcond=None)  
    cy, *_ = np.linalg.lstsq(F, sy, rcond=None)  
    return {'coef_x': cx.tolist(), 'coef_y': cy.tolist()}
```

13. Real-Time Tracking Pipeline

```
def tracking_loop():  
    cap = cv2.VideoCapture(0)  
  
    while state.running:  
        ret, frame = cap.read()  
        frame = cv2.flip(frame, 1)  
        rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)  
        mp_img = mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb)  
  
        face_result = face_det.detect(mp_img)  
  
        if face_result.face_landmarks:  
            lms = face_result.face_landmarks[0]  
            gaze_x, gaze_y = get_iris_gaze(lms)  
  
            socketio.emit('gaze_data', {
```



```

        'gazeX': gaze_x,
        'gazeY': gaze_y
    })

```

14. Hand Gesture Detection

```

def classify_hand_gesture(hand_lms):
    thumb = np.array([hand_lms[4].x, hand_lms[4].y])
    index = np.array([hand_lms[8].x, hand_lms[8].y])

    pinch_dist = np.linalg.norm(thumb - index)

    if pinch_dist < 0.05:
        return 'pinch'
    return None

```

15. API Endpoint for Control

```

@app.route('/api/start', methods=['POST'])
def start_tracking():
    state.running = True
    threading.Thread(target=tracking_loop, daemon=True).start()
    return jsonify({'status': 'started'})

```

16. WebSocket Communication

```

@socketio.on('connect')
def on_connect():
    emit('connected', {'message': 'GazeAssist ready'})

```

17. Model Auto-Download Utility

```

def ensure_model(path, url):
    if not os.path.exists(path):
        os.makedirs(os.path.dirname(path), exist_ok=True)
    try:
        urllib.request.urlretrieve(url, path)
    except Exception as e:
        raise

```

18. Calibration Application

```

def apply_calibration(model, gx, gy):
    if model is None:
        return gx, gy
    f = np.array([1, gx, gy, gx**2, gy**2, gx*gy])
    sx = float(np.dot(model['coef_x'], f))
    sy = float(np.dot(model['coef_y'], f))
    return sx, sy

```

19. Gaze Smoothing Mechanism

with state.lock:

```
a = state.SMOOTH
state.smoothed_gaze['x'] = a * gaze_x + (1-a) * state.smoothed_gaze['x']
state.smoothed_gaze['y'] = a * gaze_y + (1-a) * state.smoothed_gaze['y']
gaze_x = state.smoothed_gaze['x']
gaze_y = state.smoothed_gaze['y']
```

20. Mouse Control Logic

```
mx = int(gx_sm * (screen_w - 1))
my = int(gy_sm * (screen_h - 1))
pyautogui.moveTo(mx, my)
```

21. Hand Detection Integration

```
hand_result = hand_det.detect(mp_img)
if hand_result.hand_landmarks:
    gesture = classify_hand_gesture(hand_result.hand_landmarks[0])
```

22. Real-Time Data Emission

```
socketio.emit('gaze_data', {
    'gazeX': round(float(gaze_x), 4),
    'gazeY': round(float(gaze_y), 4),
    'faceDetected': face_detected,
    'gesture': gesture,
})
```

23. Calibration Endpoints

```
@app.route('/api/calibration/add_point', methods=['POST'])
def add_calibration_point():
    body = request.json or {}
    entry = {
        'gaze': {'x': body.get('gazeX', 0.5), 'y': body.get('gazeY', 0.5)},
        'screen': {'x': body.get('screenX', 0.5), 'y': body.get('screenY', 0.5)},
    }
    state.calibration_data.append(entry)
    return jsonify({'status': 'added'})
```

24. System Status Endpoint

```
@app.route('/api/status', methods=['GET'])
def get_status():
    return jsonify({
        'running': state.running,
        'isCalibrated': state.is_calibrated,
```

```

        'calibrationPts': len(state.calibration_data),
        'gaze':          state.smoothed_gaze,
    })

```

25. System Environment Check

```

import sys
import os
import cv2
import numpy as np

lines.append(f' Python   : {sys.version.split()[0]}')
lines.append(f' OpenCV   : {cv2.__version__}')
lines.append(f' NumPy    : {np.__version__}')

```

26. MediaPipe Dependency Check

```

try:
    import mediapipe as mp
    lines.append(f' MediaPipe: {mp.__version__}')
    mp_ok = True
except Exception as e:
    lines.append(f' MediaPipe: MISSING ({e})')
    mp_ok = False

```

27. Camera Detection & Validation

```

for idx in [0, 1, 2]:
    cap = cv2.VideoCapture(idx, cv2.CAP_DSHOW)
    if not cap.isOpened():
        lines.append(f' [{idx}] NOT available')
        continue

    ret, frame = cap.read()
    cap.release()

    if ret and frame is not None:
        lines.append(f' [{idx}] OPENED [OK]')
        found_camera = True
    else:
        lines.append(f' [{idx}] OPENED but could not grab frame [FAIL]')

```

28. Model File Verification

```

model_dir = os.path.join(os.path.dirname(os.path.abspath(__file__)),
                        'python_backend', 'models')

for fname in ['face_landmarker.task', 'hand_landmarker.task']:

```

```

path = os.path.join(model_dir, fname)
status = 'FOUND' if os.path.exists(path) else 'MISSING'

```

29. Final System Status Output

```

if found_camera:
    lines.append(' RESULT: OpenCV is working and camera is accessible.')
else:
    lines.append(' RESULT: OpenCV installed but no camera opened.')

```

30. Dataset Parsing & Metadata Extraction

```

function parseFilename(filename) {
    const parts = filename.replace('.png', '').split('_');
    if (parts.length >= 5) {
        return {
            subject: parts[0],
            session: parts[1],
            eye: parts[2],
            condition: parts[3],
            position: parseInt(parts[4]),
            sample: parts[5] ? parseInt(parts[5]) : 0,
            filename
        };
    }
    return null;
}

```

31. Mapping Dataset Positions to Calibration Grid

```

function mapPositionToCalibrationPoint(position) {
    const posStr = position.toString();
    const firstDigit = parseInt(posStr[0]);
    const secondDigit = parseInt(posStr[1] || '0');

    if (firstDigit === 2) {
        if (secondDigit <= 1) return 0;
        if (secondDigit <= 4) return 1;
        return 2;
    } else if (firstDigit === 3) {
        if (secondDigit <= 1) return 3;
        if (secondDigit <= 4) return 4;
        return 5;
    } else if (firstDigit === 4 || firstDigit === 5) {
        if (secondDigit <= 1) return 6;
        if (secondDigit <= 4) return 7;
        return 8;
    }
}

```

```

    }
    return 4;
}

```

32. Screen Coordinate Generation

```

function getScreenCoordinates(pointIndex) {
    const positions = [
        { x: 0.1, y: 0.1 }, { x: 0.5, y: 0.1 }, { x: 0.9, y: 0.1 },
        { x: 0.1, y: 0.5 }, { x: 0.5, y: 0.5 }, { x: 0.9, y: 0.5 },
        { x: 0.1, y: 0.9 }, { x: 0.5, y: 0.9 }, { x: 0.9, y: 0.9 }
    ];
    const pos = positions[pointIndex] || positions[4];
    return { x: pos.x * SCREEN_WIDTH, y: pos.y * SCREEN_HEIGHT };
}

```

33. Synthetic Eye Landmark Generation

```

function generateEyeLandmarks(pointIndex, sampleNum) {
    const screenPos = getScreenCoordinates(pointIndex);
    const gazeX = screenPos.x / SCREEN_WIDTH;
    const gazeY = screenPos.y / SCREEN_HEIGHT;

    const variation = 0.01;
    const sampleVariation = (sampleNum % 10) / 100 * variation;
    const noise = () => (Math.random() - 0.5) * variation;

    return {
        leftEye: {
            centerX: 0.45 + (gazeX - 0.5) * 0.08 + sampleVariation + noise(),
            centerY: 0.48 + (gazeY - 0.5) * 0.08 + sampleVariation + noise(),
            width: 0.05 + noise() * 0.5,
            height: 0.03 + noise() * 0.5,
            aspectRatio: 1.67 + noise() * 0.2
        },
        rightEye: {
            centerX: 0.55 + (gazeX - 0.5) * 0.08 + sampleVariation + noise(),
            centerY: 0.48 + (gazeY - 0.5) * 0.08 + sampleVariation + noise(),
            width: 0.05 + noise() * 0.5,
            height: 0.03 + noise() * 0.5,
            aspectRatio: 1.67 + noise() * 0.2
        },
        headPose: {
            pitch: noise() * 5,
            yaw: noise() * 5,
            roll: noise() * 3
        }
    };
}

```

```

    }
  };
}

```

34. Backend API Request Handler

```

function makeRequest(method, path, body = null) {
  return new Promise((resolve, reject) => {
    const url = new URL(path, API_BASE);

    const options = {
      hostname: url.hostname,
      port: url.port,
      path: url.pathname + url.search,
      method: method,
      headers: { 'Content-Type': 'application/json' }
    };

    const req = http.request(options, (res) => {
      let data = "";
      res.on('data', chunk => data += chunk);
      res.on('end', () => {
        console.log(`\n[DEBUG] Response Status: ${res.statusCode}`);
        console.log(`[DEBUG] Response Body: ${data.substring(0, 500)}`);

        try {
          const parsed = JSON.parse(data);
          if (res.statusCode >= 200 && res.statusCode < 300) {
            resolve(parsed);
          } else {
            reject(new Error(`HTTP ${res.statusCode}: ${JSON.stringify(parsed, null, 2)}`));
          }
        } catch (e) {
          reject(new Error(`Failed to parse response: ${data}`));
        }
      });
    });

    req.on('error', reject);

    if (body) {
      req.write(JSON.stringify(body));
    }
  });
}

```

```

    req.end();
  });
}

```

35. Training Pipeline Orchestrator

```

async function debugTrain() {
  console.log('🔍 DEBUG: Image Training\n');

  if (!fs.existsSync(DATASET_PATH)) {
    throw new Error(`Dataset not found: ${DATASET_PATH}`);
  }

  const imageFiles = fs.readdirSync(DATASET_PATH)
    .filter(f => f.endsWith('.png'))
    .map(f => ({ filename: f, metadata: parseFilename(f) }))
    .filter(img => img.metadata !== null);

  const health = await makeRequest('GET', '/health');
  const session = await makeRequest('POST', '/api/calibration/session', { userId:
USER_ID });

  const byPoint = {};
  for (const img of imageFiles) {
    const point = mapPositionToCalibrationPoint(img.metadata.position);
    if (!byPoint[point]) byPoint[point] = [];
    byPoint[point].push(img);
  }

  for (let point = 0; point < 9; point++) {
    const samples = (byPoint[point] || []).slice(0, 25);
    for (const img of samples) {
      const coords = getScreenCoordinates(point);
      const landmarks = generateEyeLandmarks(point, img.metadata.sample);

      await makeRequest('POST', '/api/calibration/sample', {
        sessionId: session.sessionId,
        pointIndex: point,
        screenX: coords.x,
        screenY: coords.y,
        eyeLandmarks: landmarks
      });
    }
  }
}

```

```

        await makeRequest('POST', '/api/calibration/complete', {
            sessionId: session.sessionId,
            userId: USER_ID
        });
    }
}

```

36. Script Execution Entry Point

```
debugTrain();
```

37. Gaze-to-Screen Mapping with Smoothing

```
def map_gaze_to_screen(raw_x, raw_y, tracker_state, screen_w, screen_h):
```

```
    x_history = tracker_state["x_history"]
```

```
    y_history = tracker_state["y_history"]
```

```
    x_history.append(float(raw_x))
```

```
    y_history.append(float(raw_y))
```

```
    if len(x_history) >= 20 and len(y_history) >= 20:
```

```
        min_x = max(0.0, float(np.percentile(x_history, 5)) - 0.03)
```

```
        max_x = min(1.0, float(np.percentile(x_history, 95)) + 0.03)
```

```
        min_y = max(0.0, float(np.percentile(y_history, 5)) - 0.03)
```

```
        max_y = min(1.0, float(np.percentile(y_history, 95)) + 0.03)
```

```
    else:
```

```
        min_x, max_x = 0.2, 0.8
```

```
        min_y, max_y = 0.2, 0.8
```

```
    range_x = max(max_x - min_x, 0.05)
```

```
    range_y = max(max_y - min_y, 0.05)
```

```
    norm_x = max(0.0, min(1.0, (raw_x - min_x) / range_x))
```

```
    norm_y = max(0.0, min(1.0, (raw_y - min_y) / range_y))
```

```
    target_x = norm_x * (screen_w - 1)
```

```
    target_y = norm_y * (screen_h - 1)
```

```
    last_x = tracker_state["last_x"]
```

```
    last_y = tracker_state["last_y"]
```

```
    if last_x is None or last_y is None:
```

```
        move_x, move_y = target_x, target_y
```

```
    else:
```

```
        alpha = 0.58
```

```
        move_x = last_x + (target_x - last_x) * alpha
```

```
        move_y = last_y + (target_y - last_y) * alpha
```

```
    tracker_state["last_x"] = move_x
```



```
tracker_state["last_y"] = move_y
```

```
return int(move_x), int(move_y)
```

38. MediaPipe Face Mesh Initialization

```
def create_face_mesh_detector():
```

```
    if mediapipe is None:
```

```
        return None
```

```
    if hasattr(mediapipe, "solutions") and hasattr(mediapipe.solutions, "face_mesh"):
```

```
        return mediapipe.solutions.face_mesh.FaceMesh(refine_landmarks=True)
```

```
    return None
```

39. Frame Input Handling

```
def get_frame_from_ip(url: str):
```

```
    try:
```

```
        response = requests.get(url, timeout=3)
```

```
        response.raise_for_status()
```

```
    except requests.RequestException:
```

```
        return None
```

```
    img_arr = np.array(bytearray(response.content), dtype=np.uint8)
```

```
    return cv2.imdecode(img_arr, cv2.IMREAD_COLOR)
```

40. Eye Tracking with MediaPipe

```
def process_with_mediapipe(image, face_mesh_landmarks, screen_w, screen_h,
```

```
click_cooldown, tracker_state):
```

```
    rgb_image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

```
    processed_image = face_mesh_landmarks.process(rgb_image)
```

```
    if not processed_image.multi_face_landmarks:
```

```
        return click_cooldown
```

```
    landmarks = processed_image.multi_face_landmarks[0].landmark
```

```
    for idx, point in enumerate(landmarks[474:478]):
```

```
        if idx == 1:
```

```
            mouse_x, mouse_y = map_gaze_to_screen(
                point.x, point.y, tracker_state, screen_w, screen_h
            )
```

```
            pyautogui.moveTo(mouse_x, mouse_y)
```

```
    return click_cooldown
```

41. Haar Cascade Fallback Detection

```

def process_with_haar(image, face_detector, eye_detector, screen_w, screen_h,
click_cooldown, tracker_state):
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    faces = face_detector.detectMultiScale(gray, 1.2, 5)

    if len(faces) == 0:
        return click_cooldown

    x, y, w, h = faces[0]
    roi_gray = gray[y:y+h, x:x+w]

    eyes = eye_detector.detectMultiScale(roi_gray, 1.1, 6)

    if len(eyes) > 0:
        ex, ey, ew, eh = eyes[0]
        mouse_x, mouse_y = map_gaze_to_screen(
            ex / image.shape[1], ey / image.shape[0],
            tracker_state, screen_w, screen_h
        )
        pyautogui.moveTo(mouse_x, mouse_y)

    return click_cooldown

```

42. Main Execution Loop

```

def main():
    args = parse_args()
    face_mesh_landmarks = create_face_mesh_detector()
    screen_w, screen_h = pyautogui.size()

    tracker_state = {
        "last_x": None,
        "last_y": None,
        "x_history": deque(maxlen=140),
        "y_history": deque(maxlen=140)
    }

    cap = cv2.VideoCapture(0)

    while True:
        ret, image = cap.read()
        if not ret:
            break

        image = cv2.flip(image, 1)

```

```
        if face_mesh_landmarks:
            process_with_mediapipe(image, face_mesh_landmarks, screen_w, screen_h, 0,
tracker_state)
        else:
            process_with_haar(image, None, None, screen_w, screen_h, 0, tracker_state)

cv2.imshow("Eye Controlled Mouse", image)

if cv2.waitKey(1) == 27:
    break
```

```
cap.release()
cv2.destroyAllWindows()
```

43. Program Entry Point

```
if __name__ == "__main__":
    main()
```